

Local Network Chat Protocol Specification

1. Introduction

This document describes the design and implementation of the Local Network Chat Protocol (LNC). The protocol was developed to allow communication between two applications on a local network.

The system consists of two applications:

Initiator – starts communication.

Recipient – waits for communication requests and responds to them.

The protocol uses UDP for discovery and TCP for reliable communication after a connection is established.

The current protocol version is:

LNC/1.0

2. Design Goals

The main goals of the protocol are:

- * Discover recipients on a local network.
 - * Establish reliable communication between two applications.
 - * Validate communication requests using a unique identifier (UUID).
 - * Prevent expired requests using deadlines.
 - * Support message exchange between users.
 - * Provide simple error handling.
 - * Keep the protocol easy to implement and understand.
-

3. Protocol Overview

Communication is divided into four stages:

1. Discovery
2. TCP Connection
3. Handshaking
4. Message Exchange

The Initiator first sends a UDP broadcast message containing information about the communication request.

If a Recipient recognizes its nickname, it connects to the Initiator using TCP.

After a TCP connection is established, a handshake process verifies the request.

If validation succeeds, the applications can exchange messages.

Communication follows a simplex request-response model. The Initiator sends a message and waits for a response before sending the next message.

4. Message Types

The protocol supports the following message types:

Message Type	Description
DISCOVER_UDP	discovery request
HANDSHAKE_TCP	handshake request
HANDSHAKE_OK	Successful handshake response
HANDSHAKE_REJECT	Failed handshake response
TEXT	Text message
TEXT_RESPONSE	Response to a text message
CLOSE	Connection termination request
ERROR	Error message

5. Message Format

All protocol messages use a simple text format.

General structure:

```
MESSAGE_TYPE
Header: Value
Header: Value
Message Body
```

Example:

```
TEXT
Version: LNC/1.0
Hello, I am Giorgi Rekhviashvili
```

Headers contain metadata while the body contains the message content.

6. Discovery Phase

The discovery phase uses UDP broadcast.

The Initiator broadcasts a message to the local network containing:

- * Protocol version
- * Recipient nickname
- * Request identifier (UUID)
- * Deadline
- * TCP port number

Example:

```
DISCOVER
Version: LNC/1.0
Nickname: gio
Request-Id: some uuid
Deadline: date
Tcp-Port: 5050
```

Recipients whose nickname does not match the requested nickname ignore the message.

7. TCP Connection Phase

After receiving a valid discovery message, the Recipient establishes a TCP connection to the Initiator.

The Initiator IP address is obtained from the UDP packet.

TCP is used because it provides reliable communication and ordered delivery of messages.

8. Handshake Phase

After the TCP connection is established, the Recipient sends a handshake message.

Example:

HANDSHAKE
Version: LNC/1.0
Request-Id: some uuid

The Initiator validates:

- * Protocol version
- * Request identifier
- * Deadline

If validation succeeds:

HANDSHAKE_OK
Message: Handshake accepted

If validation fails:

HANDSHAKE_REJECT
Reason: Invalid Request-Id

Possible rejection reasons include:

- * Invalid UUID
 - * Unknown UUID
 - * Expired deadline
 - * Unsupported protocol version
-

9. Message Exchange Phase

After a successful handshake, communication can begin.

The Initiator sends a text message.

Example:

TEXT
Hello

The Recipient receives the message and sends a response.

Example:

TEXT_RESPONSE
Hi

The Initiator must wait for a response before sending the next message.

This process continues until one side decides to close the connection.

10. Connection Termination

To terminate communication, the Initiator sends:

CLOSE
Conversation finished

The Recipient responds:

CLOSE
Close accepted

After receiving the response, both applications close the TCP connection.

11. Error Handling

The protocol includes basic error handling.

Errors may occur in the following situations:

- * Invalid protocol version
- * Invalid request identifier
- * Expired deadline
- * Unexpected message type
- * Malformed message format

When an error occurs, the protocol sends an ERROR or HANDSHAKE_REJECT message containing the reason for failure.

12. Assumptions and Limitations

The protocol was designed for local network communication only.

The current implementation assumes:

- * Both applications are on the same local network.
- * Only one active connection is handled at a time.
- * Messages are small enough to fit into the configured buffer size.
- * No encryption or authentication beyond UUID validation is used.

Future versions could support multiple clients, encryption, and more advanced message routing.

13. Conclusion

The Local Network Chat Protocol provides a simple solution for discovering users and exchanging messages on a local network.

The protocol combines UDP discovery with TCP communication to provide reliable message delivery. It supports discovery, handshaking, text messaging, error handling, and connection termination while remaining easy to understand and implement.